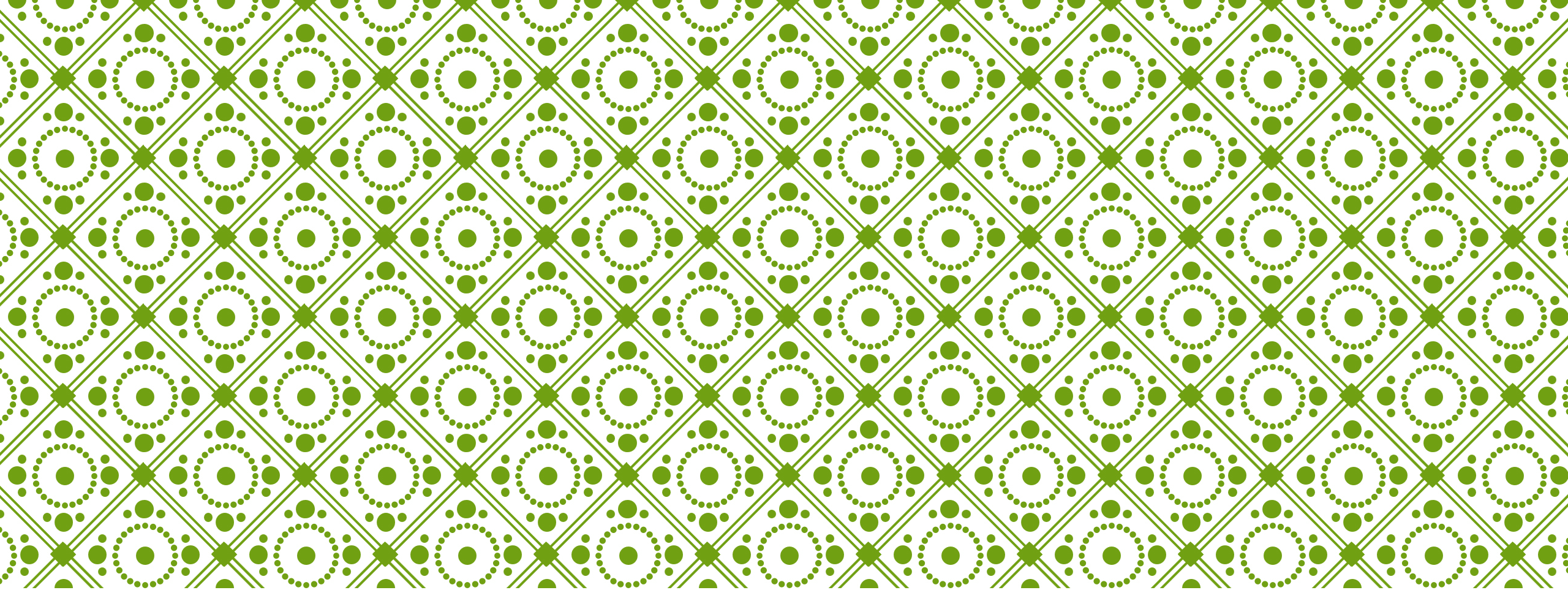


AI2002 – ARTIFICIAL INTELLIGENCE SPRING 2024 - LECTURE 13-15

Presented By: Mr. Sandesh Kumar

Few slides are taken from UC Berkley

Local Search

[These slides were created by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley. All CS188 materials are available at http://ai.berkeley.edu.]
[Updated slides from: Stuart Russell and Dawn Song]

Local Search

- The *uninformed and informed search algorithms* that we have seen are designed to explore search spaces systematically.
  - They keep one or more paths in memory and by record which alternatives have been explored at each point along the path.
  - When a goal is found, *the path to that goal also constitutes a solution to the problem*.
  - In many problems, however, the path to the goal is irrelevant.

- If ***the path to the goal does not matter***, we might consider a different class of algorithms that do not worry about paths at all.

  ➔ **local search algorithms**

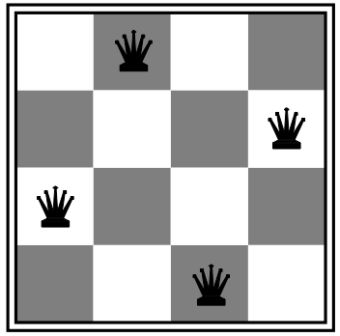
Local Search

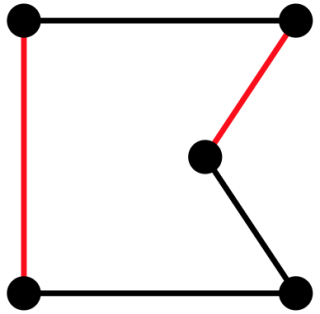
- **Local search algorithms** operate using a *single current node* and generally move only to neighbors of that node.
- **Local search algorithms** ease up on ***completeness and optimality*** in the interest of ***improving time and space complexity***?
- Although **local search algorithms** are not ***systematic***, they have ***two key advantages***:
  1. They use ***very little memory*** (usually a constant amount), and
  2. They can often find ***reasonable solutions*** in large or infinite (continuous) state spaces.

- In addition to finding goals, **local search algorithms** are useful for solving pure **optimization problems**, in which the aim is ***to find the best state*** according to an ***objective function***.
  - In optimization problems, ***the path to goal is irrelevant*** and ***the goal state itself is the solution.***
  - In some optimization problems, the ***goal is not known*** and ***the aim is to find the best state***.

Local search algorithms

- In many optimization problems, ***path*** is irrelevant; the goal state ***is*** the solution
- Then state space = set of "complete" configurations;
  find ***configuration satisfying constraints***, e.g., n-queens problem; or, find ***optimal configuration***, e.g., travelling salesperson problem

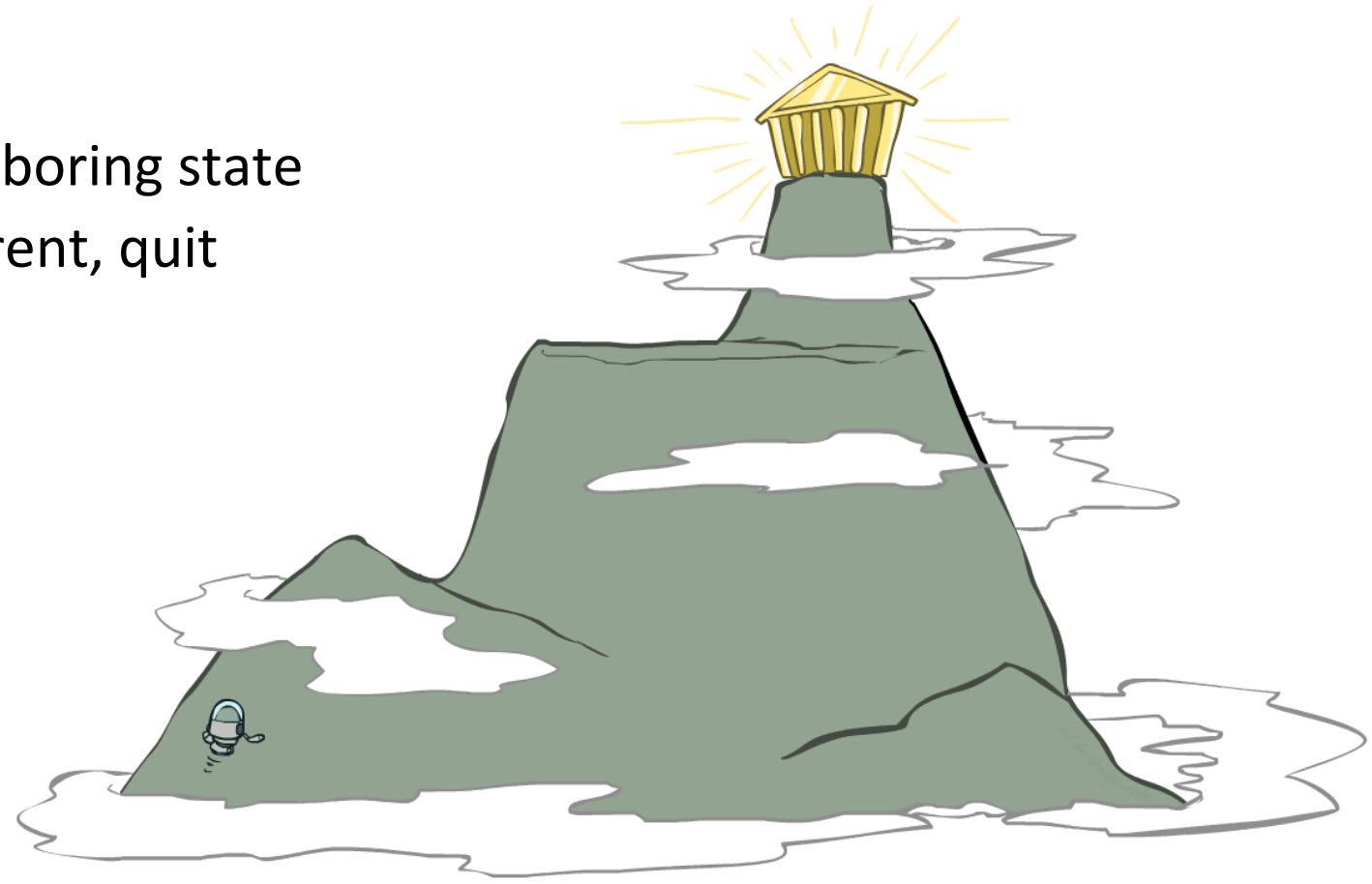




- In such cases, can use ***iterative improvement*** algorithms: keep a single "current" state, try to improve it
- Constant space, suitable for online as well as offline search
- More or less unavoidable if the "state" is yourself (i.e., learning)

Hill Climbing

- Simple, general idea:
  - Start wherever
  - Repeat: move to the best neighboring state
  - If no neighbors better than current, quit



Hill Climbing Search
(Steepest Ascent/Descent)

- At each iteration, the **hill-climbing search algorithm** moves to the ***best successor*** of the ***current node*** according to an ***objective function.***
  - Best successor is the successor with best value (highest or lowest) according to an objective function.
  - If no successors have better value than the current value, it returns.
  - It moves in direction of uphill (hill climbing).
  - It terminates when it reaches a “peak” where no neighbor has a higher value.
- The algorithm does not maintain a search tree, so the data structure for the current node need only record the state and the value of the objective function.
- **Hill climbing** does not look ahead beyond the immediate neighbors of the current state.
- **Hill climbing** is sometimes called ***greedy local search*** because it grabs a good neighbor state without thinking ahead about where to go next.
  - Greedy algorithms often perform quite well and
  - Hill climbing often makes rapid progress toward a solution.

Hill Climbing Search

(Steepest Ascent/Descent)

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

```
current ← MAKE-NODE(problem.INITIAL-STATE)
loop do
    neighbor ← a highest-valued successor of current
    if neighbor.VALUE ≤ current.VALUE then return current.STATE
    current ← neighbor
```

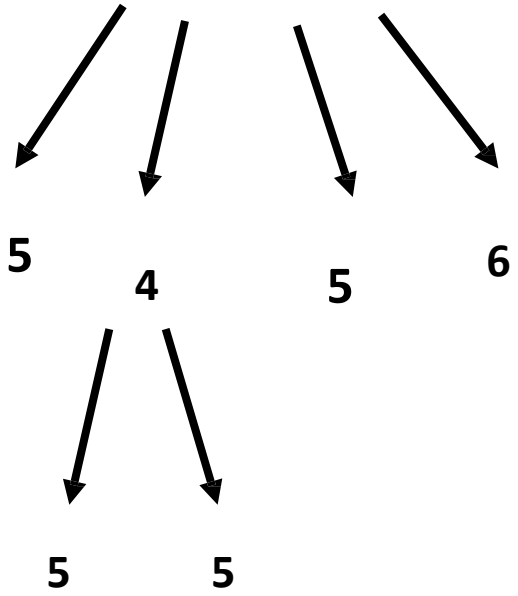
best successor

Example

5

1	2	4
5		7
3	6	8

State

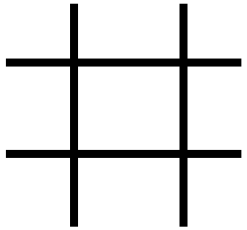


1	4	7
2	5	8
3	6	

goal

Example

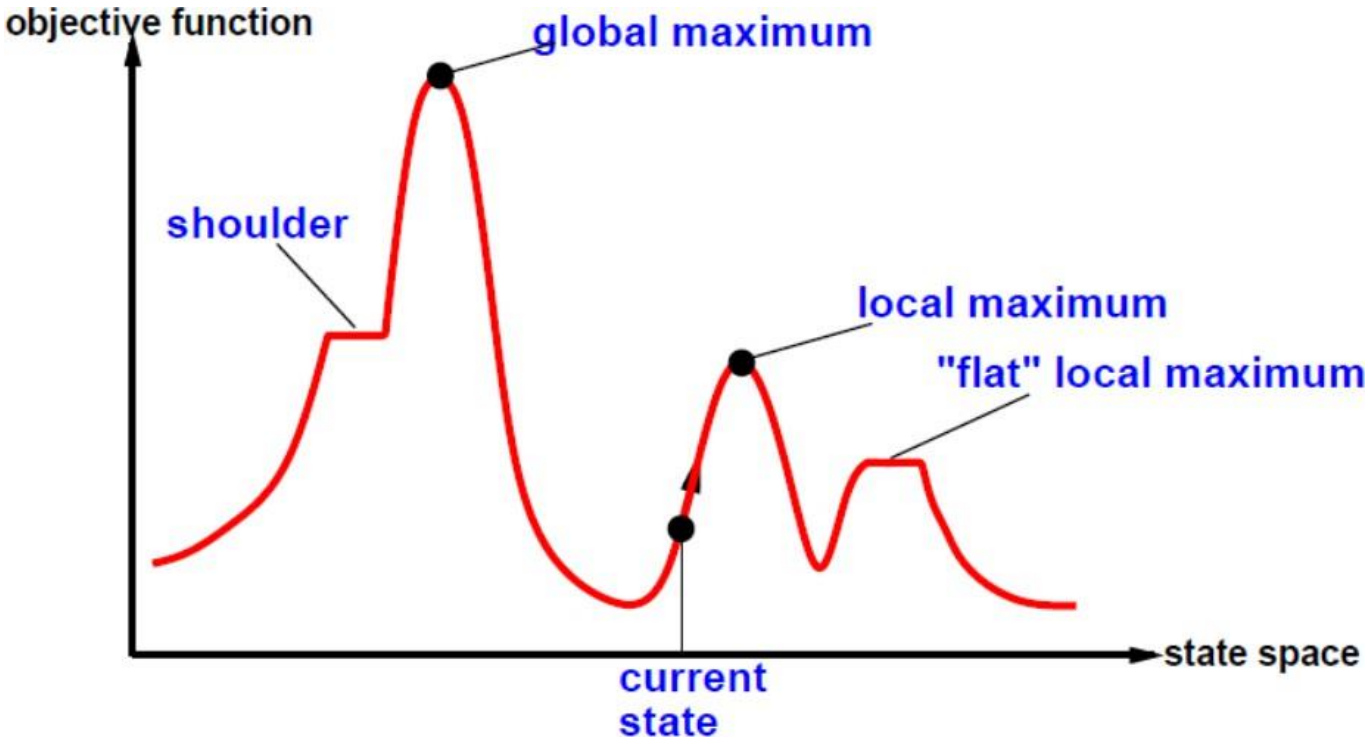
- Tic-Tac-Toe



Maximum number of winning lines

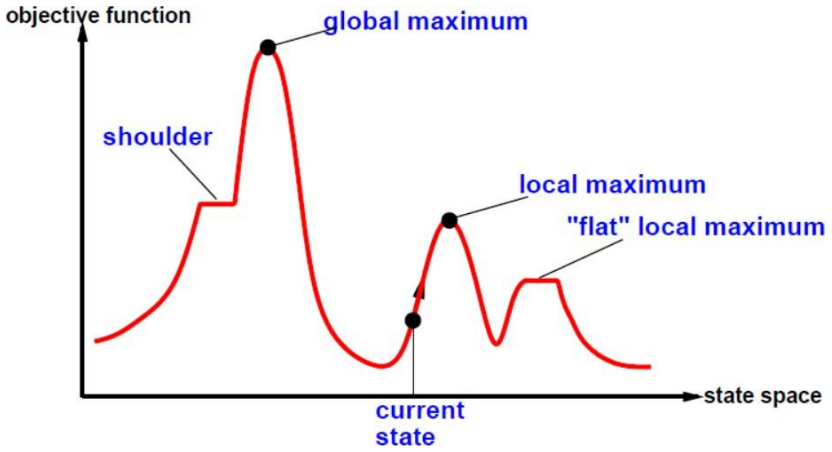
Hill Climbing Search

- To understand local search, it is useful to consider the ***state-space landscape.***
- The aim is to find the highest peak - **a global maximum**.
- Hill-climbing search modifies the current state to try to improve it.

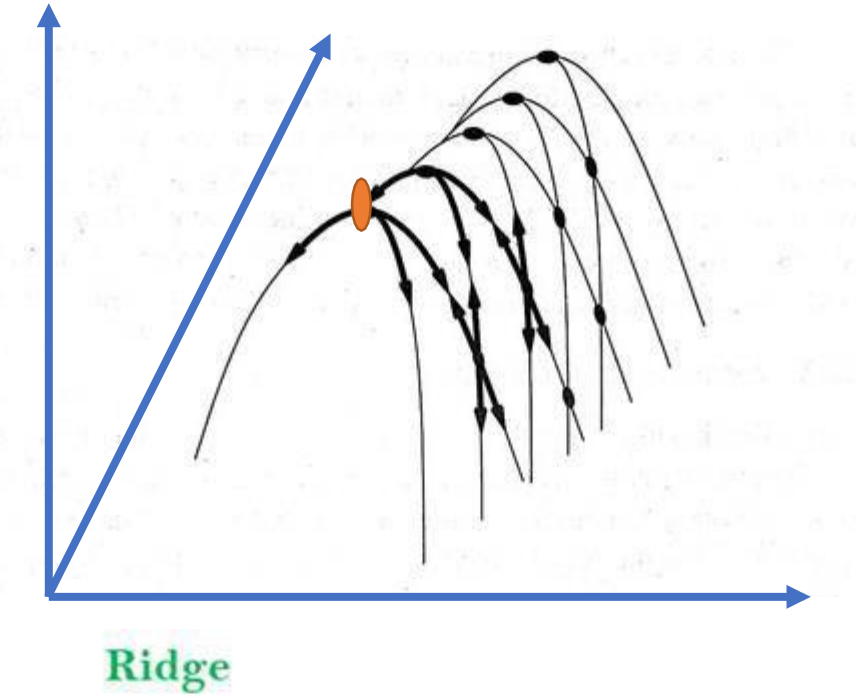


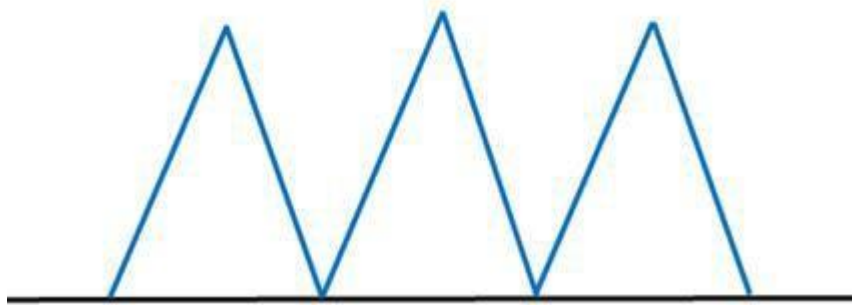
Hill Climbing Search

- A **local maximum** is a peak that is higher than each of its neighboring states but lower than the **global maximum**.
  - Hill-climbing algorithms that reach the vicinity of a local maximum will be drawn upward toward the peak but will then be stuck with nowhere else to go.
- A **plateau** is a **flat area** of the state-space landscape. It can be a **flat local maximum**, from which no uphill exit exists, or a **shoulder**, from which progress is possible.
  - A hill-climbing search might get lost on the plateau.
  - **Random sideways moves** can escape from ***shoulders*** but they loop forever on ***flat maxima.***



Problems in Hill Climbing…





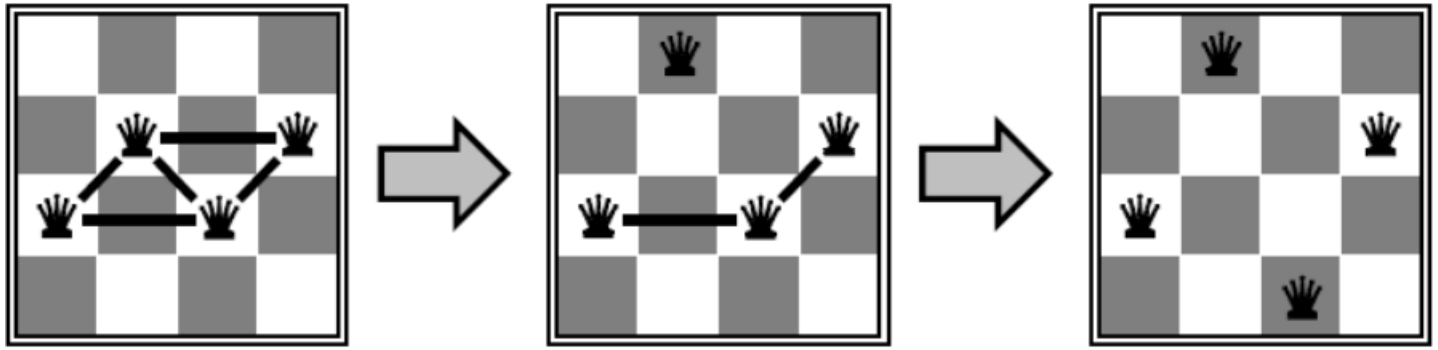
3. Ridges: A ridge is a special form of the local maximum. It has an area which is higher than its surrounding areas, but itself has a slope, and cannot be reached in a single move.

Hill Climbing Example

n-queens

- Put n queens on an n × n board with no two queens on the same row, column, or diagonal
- Move a queen to reduce number of conflicts.

➔ **Objective function: number of conflicts (no conflicts is global minimum)**

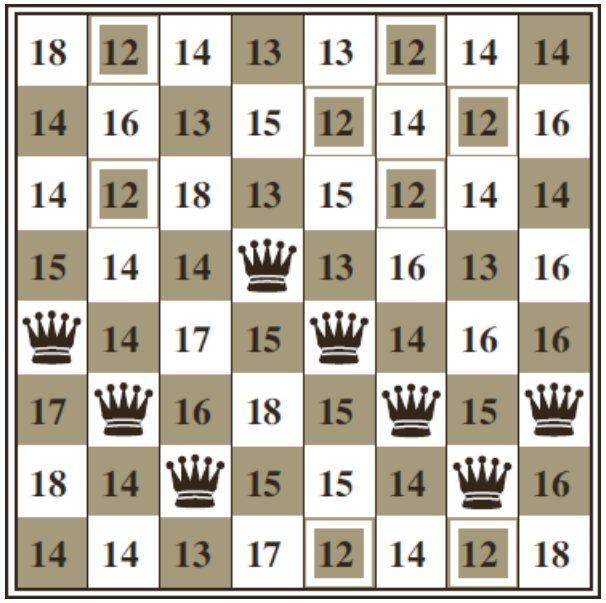


- The successors of a state are all possible states generated by moving a single queen to another square in the same column (so each state has n*(n-1) successors).
- The heuristic cost function h is the number of pairs of queens that are attacking each other, either directly or indirectly.
- The global minimum of this function is zero, which occurs only at perfect solutions.

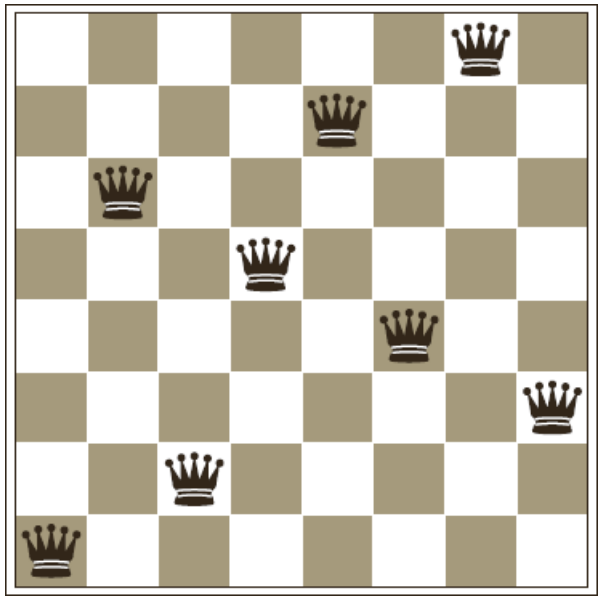
Hill Climbing Example

n-queens

- Hill Climbing may NOT reach to a goal state for n-queens problem.



➔ five steps to reach this state



- An 8-queens state with heuristic cost estimate h=17
- The value of h for each possible successor obtained by moving a queen within its column.
- The best moves are marked with value 12.

- A local minimum in the 8-queens state space; the state has h=1
- but every successor has a higher cost.
- Hill Climbing will stuck here

Hill Climbing Example

n-queens

- Starting from a randomly generated 8-queens state, **steepest-ascent hill climbing** gets stuck 86% of the time, solving only 14% of problem instances.

- The Hill Climbing algorithm halts if it reaches a **plateau.**
  - One possible solution is to allow **sideways move** in the hope that the **plateau** is really a **shoulder**.
  - If we always allow sideways moves when there are no uphill moves, an infinite loop will occur whenever the algorithm reaches a flat local maximum that is not a shoulder.
  - One common solution is to put a limit on the number of consecutive sideways moves allowed.
  - For example, we could allow up to 100 consecutive sideways moves in the 8-queens problem. This raises the percentage of problem instances solved by hill climbing from 14% to 94%.

Hill Climbing Example

8-puzzle

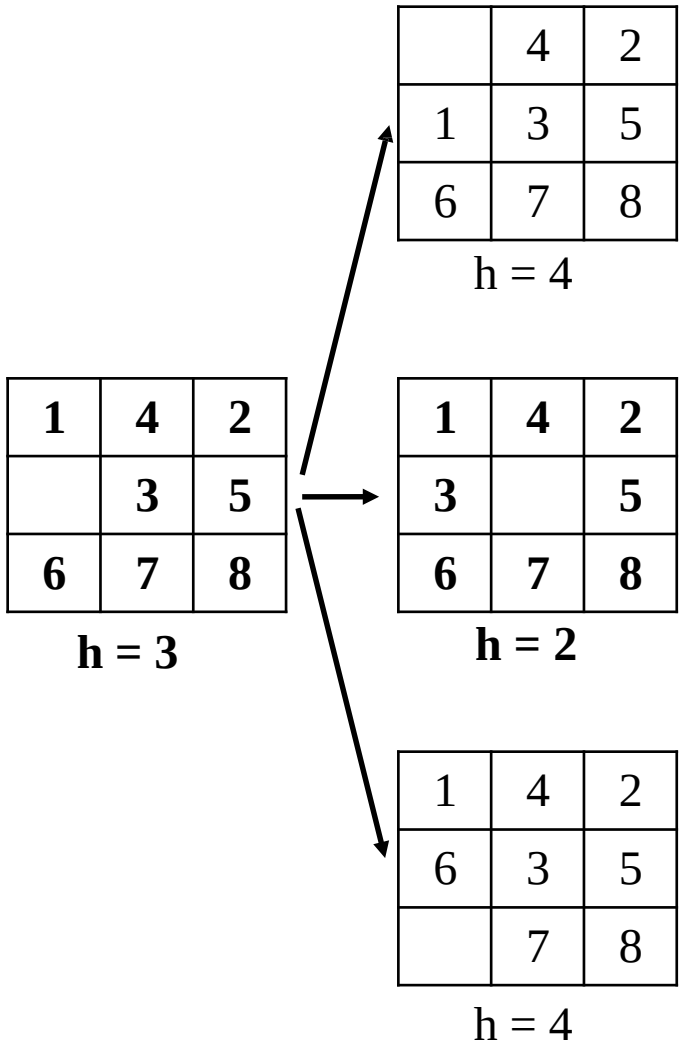
Heuristic function is Manhattan Distance

1	**4**	**2**
	3	**5**
6	**7**	**8**

h = 3

Hill Climbing Example

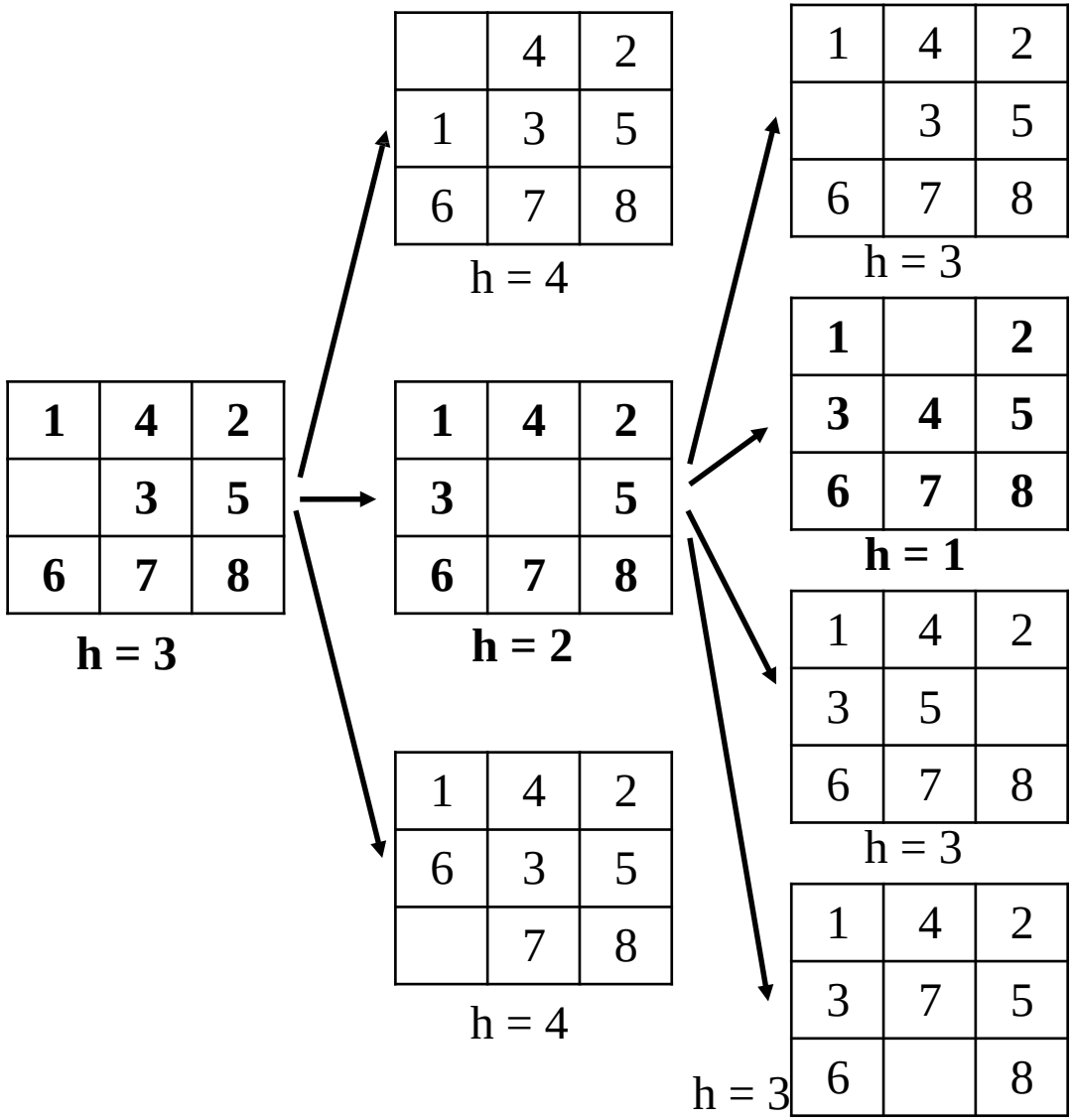
8-puzzle



Heuristic function is Manhattan Distance

Hill Climbing Example

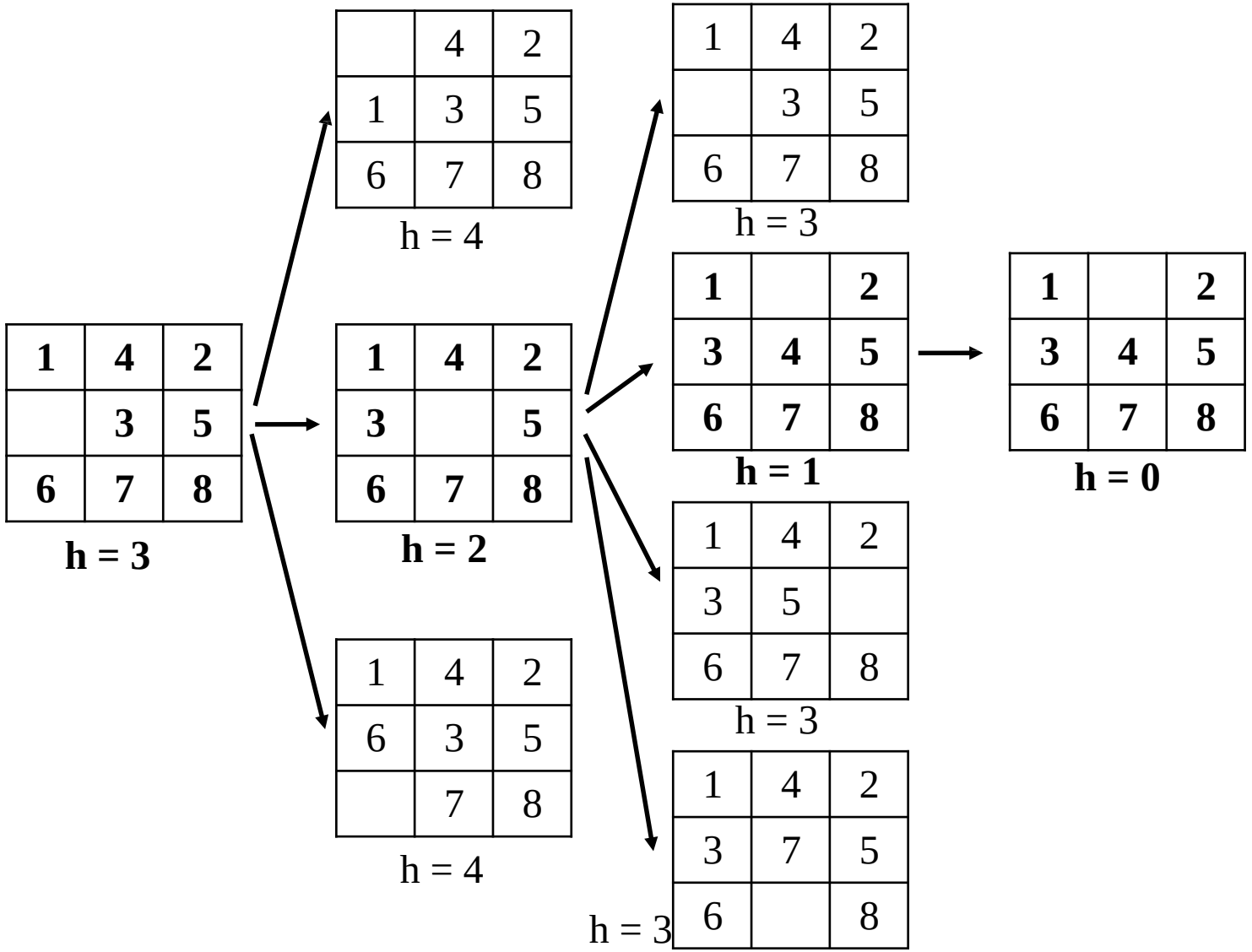
8-puzzle



Heuristic function is Manhattan Distance

Hill Climbing Example

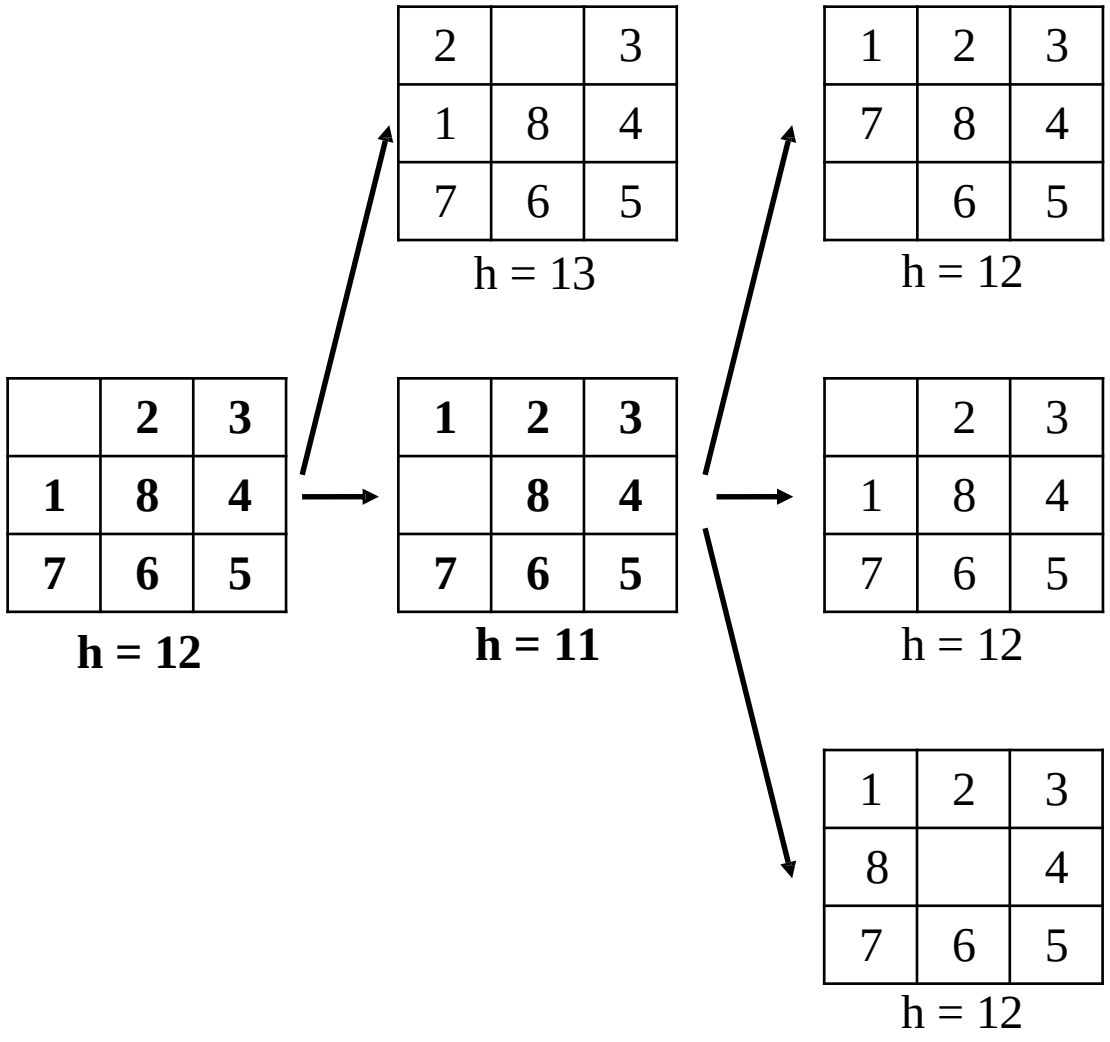
8-puzzle: a solution case



Heuristic function is Manhattan Distance

Hill Climbing Example

8-puzzle: stuck at local maximum



Heuristic function is Manhattan Distance

We are stuck with a local maximum.

Hill Climbing

- Hill Climbing is **NOT complete**.
- Hill Climbing is **NOT optimal**.

- **Why use local search?**
  - Low memory requirements – usually constant
  - Effective – Can often find good solutions in extremely large state spaces
  - Randomized variants of hill climbing can solve many of the drawbacks in practice.

- Many variants of hill climbing have been invented.

Simulated Annealing

- A **hill-climbing algorithm** that never makes "downhill" moves toward states with lower value (or higher cost) is guaranteed to be **incomplete**, because ***it can get stuck on a local maximum***.
  - In contrast, a **purely random walk**—that is, moving to a successor chosen uniformly at random from the set of successors—is **complete but extremely inefficient**.
  - Therefore, it seems reasonable to ***combine hill climbing with a random walk in some way that yields both efficiency and completeness***.
- **Idea:** ***escape local maxima by allowing some "bad" moves but gradually decrease their size and frequency.***
- The **simulated annealing algorithm**, a version of ***stochastic hill climbing*** where some downhill moves are allowed.
- **Annealing:** the process of gradually cooling metal to allow it to form stronger crystalline structures
- **Simulated annealing algorithm:** gradually "cool" search algorithm from Random Walk to First-Choice Hill Climbing

Simulated Annealing

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state
  inputs: *problem*, a problem
    schedule, a mapping from time to "temperature"

  current ← MAKE-NODE(*problem*.INITIAL-STATE)
  for $t = 1$ **to** ∞ **do**
    $T \leftarrow schedule(t)$
    if $T = 0$ **then return** *current*
    next ← a randomly selected successor of *current*
    $\Delta E \leftarrow next.\text{VALUE} - current.\text{VALUE}$
    if $\Delta E > 0$ **then** *current* ← *next*
    else *current* ← *next* only with probability $e^{\Delta E/T}$

- **Downhill moves** are accepted readily early in the annealing schedule and then less often as time goes on.
- The **schedule** input determines the value of the temperature T as a function of time.

Simulated Annealing

- Instead of picking the best move, Simulated Annealing picks a random move.
  - If the move improves the situation, it is always accepted.
  - Otherwise, the algorithm accepts the move with some probability less than 1.
- The probability decreases exponentially with the “badness” of the move—the amount ΔE by which the evaluation is worsened.
- The probability also decreases as the “temperature” T goes down: “bad” moves are more likely to be allowed at the start when T is high, and they become more unlikely as T decreases.
- **If the schedule lowers T slowly enough, the algorithm will find a best state with probability approaching 1.**

- Simulated Annealing is widely used in VLSI layout and airline scheduling.

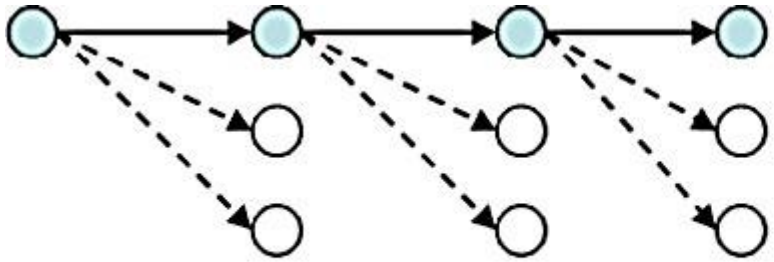
Local Beam Search

- The **local beam search algorithm** keeps track of ***k states rather than just one***.
  - It begins with k randomly generated states.
  - At each step, all the successors of all k states are generated.
  - If any one is a goal, the algorithm halts.
  - Otherwise, it selects the k best successors from the complete list and repeats.

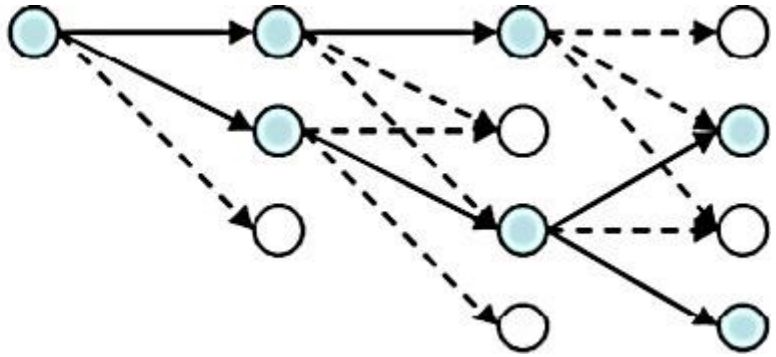
- The **local beam search algorithm** is **not** the same as ***k searches run in parallel***!
  - In a **local beam search**, searches that find good states recruit other searches to join them.
  - In a random-restart search, each search process runs independently of the others.

Local Beam Search

- Start with k randomly generated states (ß= 2 or 3)
- At each iteration, all the successors of all k states are generated
- If any one is a goal state, stop; else select the k best successors from the complete list and successors from the complete list and repeat



Greedy Local



Local Beam

Stochastic Beam Search

- **Problem:** quite often, all k states end up on same local hill in ***local beam search algorithm***
- **Idea:** choose k successors randomly, biased towards good ones

  ➔ **Stochastic Beam Search**

- Instead of choosing the best k from the pool of candidate successors, **stochastic beam search** chooses k successors at random, with the probability of choosing a given successor being an increasing function of its value.

- **Stochastic beam search** bears some resemblance to the process of natural selection, whereby the "successors" (offspring) of a "state" (organism) populate the next generation according to its "value" (fitness).

Genetic Algorithms

- A **genetic algorithm (GA)** is a variant of ***stochastic beam search*** in which successor states are generated by combining two parent states rather than by modifying a single state.
- Like beam searches, **GAs** begin with a set of ***k randomly generated states***, called the **population**.
- Each state, or **individual**, is represented as a string over a finite alphabet—most commonly, a string of 0s and 1s.
  - An 8-queens state must specify the positions of 8 queens, each in a column of 8 squares, and so it requires 8× log2 8=24 bits.
  - Alternatively, the state could be represented as 8 digits, each in the range from 1 to 8.
- Each state is rated by an ***objective function***, or (in GA terminology) the **fitness function**.
- Pairs of individuals are randomly selected for ***reproduction***.
- From each pair new offsprings (children) are generated using **crossover** operation.
  - A crossover point is chosen randomly from the positions in the string.
  - The offsprings are created by crossing over the parent strings at the crossover point.
- Each child is subject to random **mutation** with a small independent probability.

GeneticAlgorithm

Introduced in the 1970s by John Holland at University of Michigan

- begin with *k* randomly generated states (population)
- each state (individual) is a string over some alphabet (chromosome)
- fitness function (bigger number is better)
- crossover
- mutate (evolve?)

Nature of Computer Mapping

Nature	Computer
Population	Set of solutions.
Individual	Solution to a problem.
Fitness	Quality of a solution.
Chromosome	Encoding for a Solution.
Gene	Part of the encoding of a solution.
Reproduction	Crossover

Computational Model

Step 1: Representing individuals.

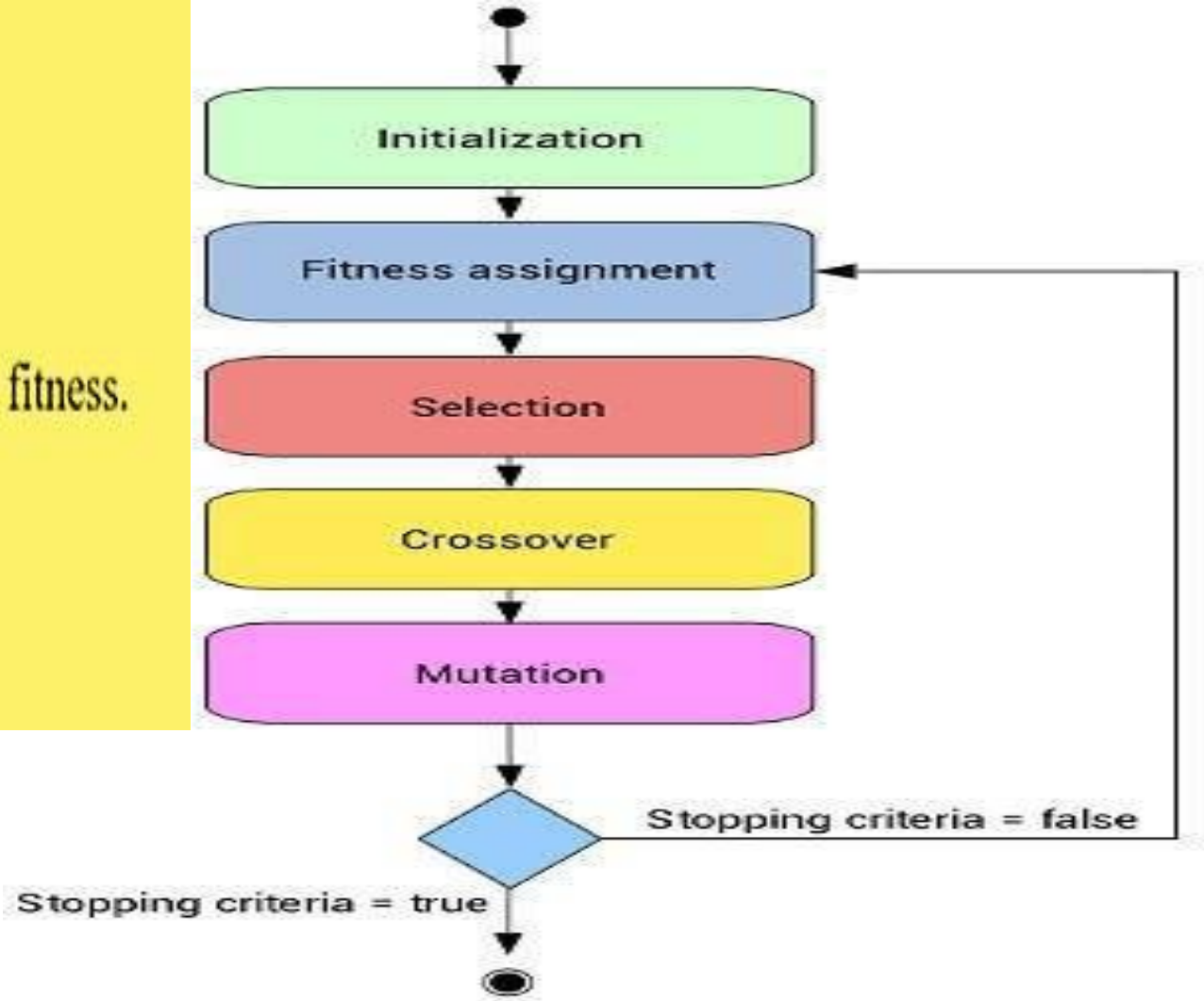
Step 2: Generating an initial Population.

Step 3: Applying a Fitness Function.

Step 4: Selecting parents for mating in accordance to their fitness.

Step 5: Crossover of parents to produce new generation.

Step 6: Mutation of new generation to bring diversity.



Encoding

*The process of representing the solution in the form of a **string** that conveys the necessary information.*

- **Binary Encoding** – Most common method of encoding. Chromosomes are strings of 1s and 0s and each position in the chromosome represents a particular characteristic of the problem.

  Permutation Encoding – Useful in ordering problems such as the Traveling Salesman Problem (TSP). Example. In TSP, every chromosome is a string of numbers, each of which represents a city to be visited.

- **Value Encoding** – Used in problems where complicated values, such as real numbers, are used and where binary encoding would not suffice.

Crossover

It is the process in which two chromosomes (strings) combine their genetic material (bits) to produce a new offspring which possesses both their characteristics.

- *Two strings are picked from the mating pool at random to cross over.*
- *The method chosen depends on the Encoding Method.*

Crossover

- **Single Point Crossover-** A random point is chosen on the individual chromosomes (strings) and the genetic material is exchanged at this point.

Chromosome1	11011 | 00100110110
Chromosome 2	11011 | 11000011110
Offspring 1	11011 | 11000011110
Offspring 2	11011 | 00100110110

Crossover

- **Two-Point Crossover-** Two random points are chosen on the individual chromosomes (strings) and the genetic material is exchanged at these points.

Chromosome1	11011 | 00100 | 110110
Chromosome 2	10101 | 11000 | 011110
Offspring 1	10101 | 00100 | 011110
Offspring 2	11011 | 11000 | 110110

NOTE: These chromosomes are different from the last example.

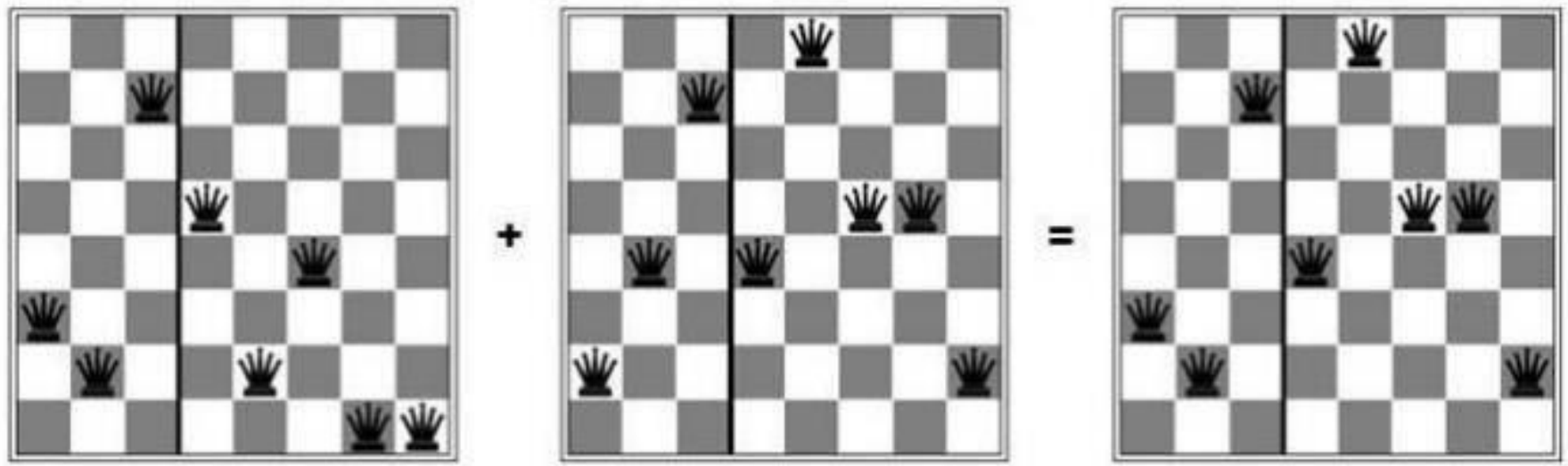
Crossover

- **Uniform Crossover-** Each gene (bit) is selected randomly from one of the corresponding genes of the parent chromosomes.

Chromosome1	11011 | 00100 | 110110
Chromosome 2	10101 | 11000 | 011110
Offspring	10111 | 00000 | 110110

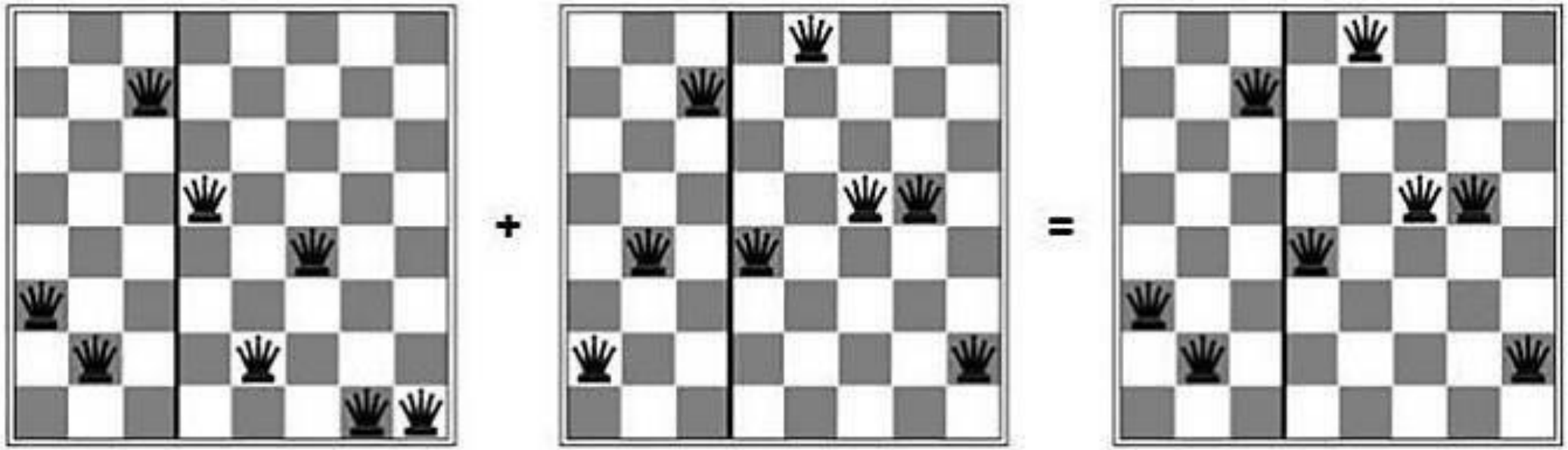
NOTE: Uniform Crossover yields ONLY 1 offspring.

Genetic algorithms:8-queens



GeneticAlgorithm

- Fitness function= Pair of non-attacking queens
- That way higher scores are better

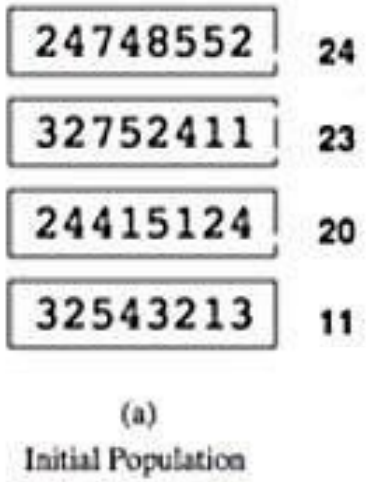


23 fitness

24748552 string

Fitness function

Represent states and compute fitness function.



(a)
Initial Population

Probability= 24+23+20+11= 78

Probability

Compute probability of being chosen (from fitness function).

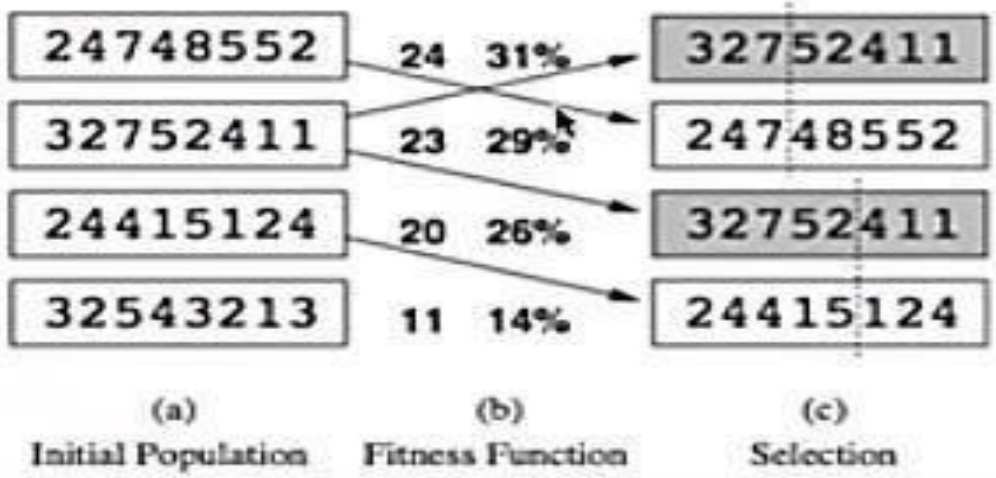
24748552	24	31%
32752411	23	29%
24415124	20	26%
32543213	11	14%

(a)
Initial Population

24/78= 0.307 Normalize 0.307× **100 = 30.7%**
chance of being chosen probably

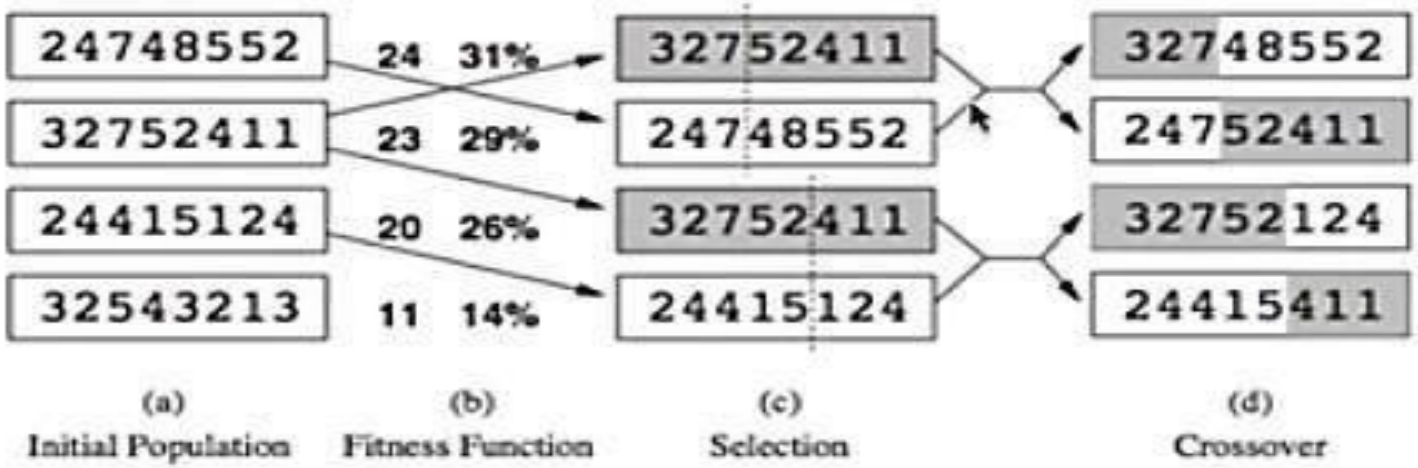
Reproduction

Randomly choose two pairs to reproduce based on probabilities. Pick a crossover point per pair.

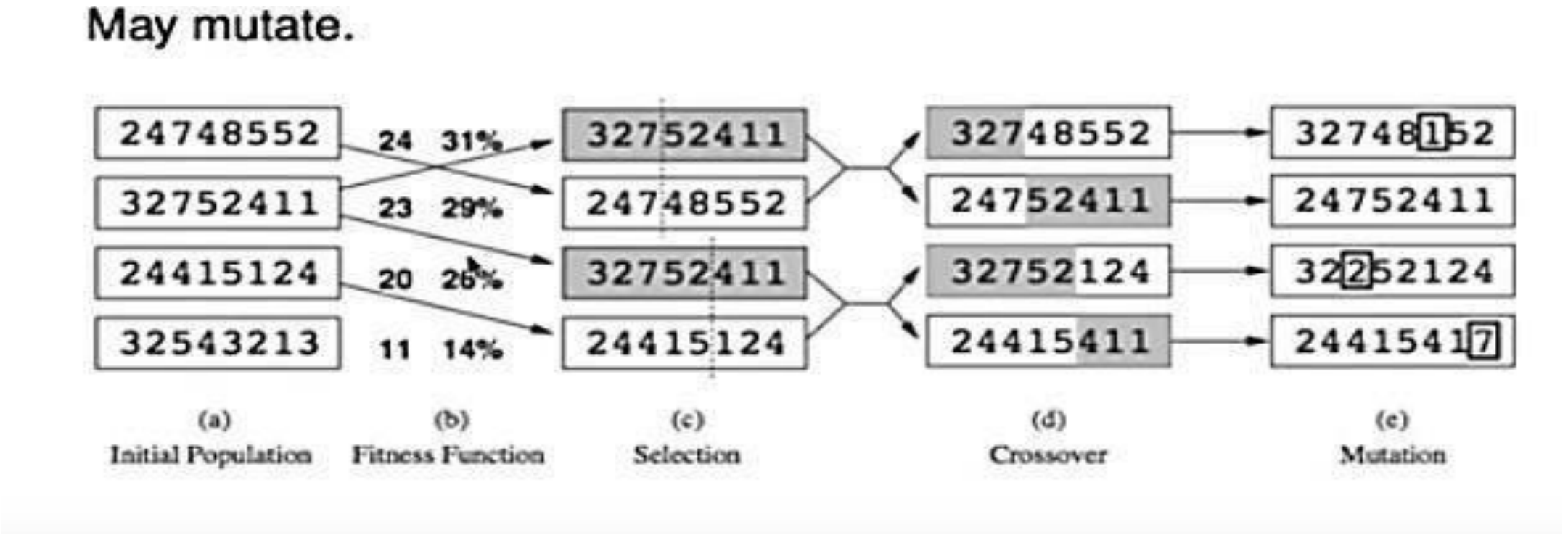


Crossover

Crossover, produce offspring.



Mutation


May mutate.
24748552
32752411
24415124
32543213
24 31%
23 29%
20 26%
11 14%
32752411
24748552
32752411
24415124
32748552
24752411
32752124
24415411
32748152
24752411
32252124
24415417
(a)
Initial Population
(b)
Fitness Function
(c)
Selection
(d)
Crossover
(e)
Mutation

Knapsack Problem

- Let's say, you are going to spend a month in the wilderness. Only thing you are carrying is the backpack which can hold a maximum weight of **30 kg**. Now you have different survival items, each having its own "Survival Points" (which are given for each item in the table). So, your objective is maximize the survival points.
- Here is the table giving details about each item.

ITEM	WEIGHT	SURVIVAL POINTS
SLEEPING BAG	15	15
ROPE	3	7
POCKET KNIFE	2	10
TORCH	5	5
BOTTLE	9	8
GLUCOSE	20	17

- Choose an initial population of four. Then choose best three subsequently (Encoding hint: 1 if item is chosen 0 otherwise)
- Decide crossover point.
- Chance of mutation = 50 items in 100 generations.
- Step by step solution ahead

① Initial population (Initially I am taking 4 states chosen randomly which have weight ≤ 30)

100110
000011
011101
111100

② Second step : Selection / Fitness function : The one with highest survival point is to be selected (i.e best 3 are the ones which have highest survival point).

100110	→	15+0+0+5+8+0 = 28 ✓
000011	fitness →	0+0+0+0+8+17 = 25
011101	function →	0+7+10+5+0+17 = 39 ✓
111100	→	15+7+10+5+0+0 = 37 ✓

Best three are states 1, 3 & 4

③ Selection : 1, 3 & 4 states

Combination : [1 & 3] & [3 & 4]

1 & 3

1 0 0 1 1 0

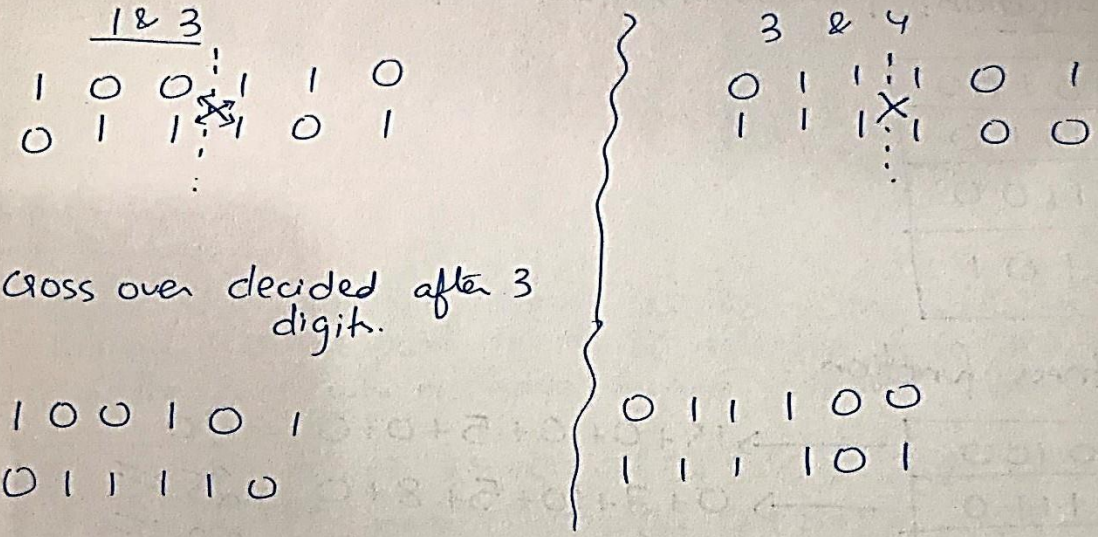
0 1 1 1 0 1

0 1 1 1 0 1

1 1 1 1 0 0

④ ~~Next~~

④ Now cross over will be applied.



Cross over decided after 3 digit.

1 0 0 1 0 1
0 1 1 1 1 0

0 1 1 1 0 0
1 1 1 1 0 1

⑤ Now mutation (we had 50 items in 100 generations).

Because in question we are told to perform 3 iterations; one of them is initialization iteration & 2 are generation thereby.

50 items —— 100 generation

x —— 2 "

$x = 1$

Thereby one bit mutation will only take place in this entire 3 iteration phase.

Our cross over brought 4 new states.

mutate to

1 0 0 1 0 (1) 0.

0 1 1 1 1 0

0 1 1 1 0 0

1 1 1 1 0 1

Now if you want you can mutate one bit in this iteration, else do it in any ~~two~~ other iteration. I have mutated in this iteration thereby output of first iteration is

1 0 0 1 0 0
0 1 1 1 1 0
0 1 1 1 0 0
1 1 1 1 0 1

Now this output of first iteration will be the input for second iteration.

Second iteration

① Initialization: Previous iteration output.

100100
011110
011100
111101

② Fitness function:

100100	→ 15+0+0+5+0+0 = 20
011110	→ 0+7+10+5+8+0 = 25 =
011100	→ 0+7+10+5+0+0 = 22 =
111101	→ 15+7+10+5+0+17 = 54 =

③ Selection 2 & 3 & 4

④ Cross over

2 & 3

011|110
011|100

011100
011110

3 & 4

011|100
111|101

011101
111100

Crossover results in

0 1 1 1 0 0
0 1 1 1 1 0
0 1 1 1 0 1
1 1 1 1 0 0

No mutation now because I have already done in previous iteration & only one bit mutation was allowed in the question

* Point to notice: See state 3 in this crossover that's exactly holding weight 30 & having survival 39 which is till now the best state as we can also hold more items & survival is also more.

Now as no mutation is done we will input the crossover output of this iteration to third iteration.

That means.

(3) Third iteration

Initialization

0 1 1 1 0 0
0 1 1 1 1 0
0 1 1 1 0 1
1 1 1 1 0 0

→ Now rest of the steps will be done & the end.

You can do the third iteration on your own now. First two iterations have been done by me.

Properties

- Work well for mixed (continuous and discrete) problems
- They are less suceptible to get stuck at local optima
- Computationally expensive
- However, easy to perform in parallel
- No math in the process. The objective (fitness) function may be hard

Genetic Algorithms

A genetic algorithm: *each mating of two parents produces only one offspring, not two.*

```
function GENETIC-ALGORITHM(population, FITNESS-FN) returns an individual
  inputs: population, a set of individuals
          FITNESS-FN, a function that measures the fitness of an individual

  repeat
      new_population ← empty set
      for i = 1 to SIZE(population) do
          x ← RANDOM-SELECTION(population, FITNESS-FN)
          y ← RANDOM-SELECTION(population, FITNESS-FN)
          child ← REPRODUCE(x, y)
          if (small random probability) then child ← MUTATE(child)
          add child to new_population
      population ← new_population
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to FITNESS-FN
```

```
function REPRODUCE(x, y) returns an individual
  inputs: x, y, parent individuals

  n ← LENGTH(x); c ← random number from 1 to n
  return APPEND(SUBSTRING(x, 1, c), SUBSTRING(y, c + 1, n))
```